

Resumen para Parcial

Introducción a los ORM

Un **ORM (Object-Relational Mapper)** busca resolver, fundamentalmente, la **Diferencia de Impedancia (Impedance Mismatch)**. Este problema es la desconexión estructural y conceptual entre el modelo de objetos y el modelo relacional.

Al intentar persistir objetos directamente con JDBC puro se presentan ciertos inconvenientes como:

- *"Contaminación" del código y dificultad de mantenimiento*
- *Gestión manual de la jerarquía e identidad*
- *Complejidad en la carga de grafos de objetos (Lazy Loading)*

Ventajas y Desventajas del uso de un ORM en una solución

Característica	Persistencia con JDBC puro	Persistencia con ORM (Hibernate/JPA)
Código de consulta	SQL manual embebido en Strings (difícil de mantener).	Consultas orientadas a objetos (JPQL/HQL) o métodos de interfaz.
Mapeo de datos	Manual (mapeo de <code>ResultSet</code> a objeto fila por fila).	Automático (mapeo de atributos a columnas mediante metadatos).
Herencia	No soportada nativamente; requiere lógica manual compleja.	Soporte nativo mediante estrategias como <code>SINGLE_TABLE</code> o <code>JOINED</code> .
Relaciones	Se resuelven mediante <code>JOINS</code> explícitos en cada consulta.	Navegación directa entre referencias de objetos en Java.
Carga de grafos	El programador decide explícitamente qué traer de la BD.	Carga automática con opciones de <code>LAZY</code> o <code>EAGER</code> .
Gestión de cambios	Requiere sentencias <code>UPDATE</code> explícitas para cada cambio.	<i>Dirty Checking</i> : el ORM detecta y sincroniza cambios automáticamente al hacer <i>commit</i> .
Independencia	Atado fuertemente al dialecto SQL del motor específico.	Alta; el ORM traduce la lógica al dialecto configurado (MySQL, Oracle, etc.).

JPA e Hibernate

JPA es el **estándar o especificación oficial de persistencia** para la plataforma Java. Se define como un **contrato de persistencia estándar** que establece un conjunto de reglas y definiciones para gestionar cómo los objetos de una aplicación Java se almacenan y recuperan de una base de datos.

Proporciona una **API y un conjunto de anotaciones** que permiten a los desarrolladores describir el mapeo entre el modelo de objetos y el modelo relacional de forma declarativa.

Se dice que es una **especificación** porque JPA solo define el **"qué"** (las interfaces, anotaciones y reglas del contrato), pero no proporciona el código ejecutable que realiza el trabajo pesado de conectarse a la base de datos y ejecutar el SQL, para que JPA funcione, necesita de una **implementación concreta** que traduzca esas definiciones en acciones reales.

Técnicamente no se puede usar JPA "solo", porque JPA es una especificación y requiere de un **proveedor de persistencia** (Provider) que contenga el código ejecutable. Sin embargo, **sí se puede usar JPA sin Hibernate específicamente**, siempre que se utilice otra implementación que cumpla con el estándar.

Hibernate sin JPA obvio que se puede usar, es más, Hibernate existía mucho antes de que se creara el estándar JPA.

SessionFactory

La **SessionFactory** es uno de los componentes centrales de la arquitectura de Hibernate. Se define como un objeto **inmutable que actúa como un mapa o caché de mapeos compilados** para una base de datos específica.

Implementa 2 patrones de diseño: **Factory y Singleton**.

Existen razones técnicas y de performance fundamentales para mantener una única instancia durante todo el ciclo de vida de la aplicación:

- **Alto costo de inicialización (Heavyweight object)**
- **Gestión del Second Level Cache**
- **Seguridad de hilos (Thread-Safety)**
- **Administración de conexiones**

Formas de obtener una sesion

En la práctica se usó `getCurrentSession()`

<code>openSession()</code>	<code>getCurrentSession()</code>
Este método siempre abre una nueva sesión en la base de datos,	Este método busca una sesión que ya esté vinculada al contexto actual (usualmente

<code>openSession()</code>	<code>getCurrentSession()</code>
independientemente de si ya existe una activa en el contexto actual. El desarrollador tiene el control total sobre su ciclo de vida: debe abrirla, iniciar la transacción, confirmar los cambios y, fundamentalmente, cerrarla manualmente para evitar fugas de memoria	asociado a la transacción en curso). Si no existe una, la crea; si ya existe, devuelve la misma instancia. Su ciclo de vida está íntimamente ligado al de la transacción: cuando la transacción termina (ya sea por <i>commit</i> o <i>rollback</i>), la sesión se cierra automáticamente
Cerrar sesión con <code>openSession()</code>	Cerrar sesión con <code>getCurrentSession()</code>
La responsabilidad recae enteramente en el desarrollador . Es obligatorio incluir una llamada a <code>session.close()</code> para liberar la conexión al pool	La responsabilidad es de Hibernate (o del framework de soporte como Spring). Al finalizar la unidad de trabajo (la transacción), el motor detecta el cierre y libera los recursos de forma automática, reduciendo errores humanos

Comparación

Aspecto	JPA	Hibernate
Tipo	Especificación	Implementación
Define interfaces y anotaciones	Si	Si (Implementa las de JPA y añade anotaciones propias adicionales)
Proporciona implementaciones concretas	No	Si
Genera SQL a partir de consultas JPQL/HQL	No	Si
Maneja el Persistence Context	Define el contrato y comportamiento esperado del contexto	Lo gestiona físicamente (usualmente mediante el objeto <code>Session</code>)

Ciclo de Vida de las Entidades en Hibernate

Transient

Es el estado inicial de un objeto recién instanciado en la memoria de la aplicación.

- **Descripción:** El objeto existe en la RAM pero no tiene una identidad en la base de datos (Clave Primaria) ni está asociado a ninguna sesión de Hibernate.

- **Transición:** Se entra en este estado mediante el operador **new** (ej. `Purchase p = new Purchase()`).

Managed / Persistent

En este estado, el objeto está bajo el control total del motor de persistencia.

- **Descripción:** La entidad tiene un identificador único (ID) y está vinculada a una `Session` activa. Hibernate utiliza un mecanismo de **Dirty Checking** para detectar automáticamente cualquier cambio en los atributos y sincronizarlos con la base de datos al realizar el **flush** o el **commit** de la transacción.
- **Transiciones hacia Managed:**
 - Desde **Transient**: Al invocar `session.persist()`, `session.save()` o mediante **persistencia por alcance** (si un objeto gestionado referencia a uno transitorio y hay cascada activa).
 - Desde **Detached**: Al invocar `session.merge()` (que devuelve una copia gestionada), `session.update()` o `session.saveOrUpdate()`.
 - Desde la Base de Datos: Al recuperar el objeto mediante una **consulta (HQL/JPQL)** o métodos como `session.get()` o `session.load()`.

Detached

El objeto sigue existiendo en memoria y posee su ID, pero Hibernate ya no rastrea sus cambios.

- **Descripción:** Ocurre cuando el vínculo entre el objeto y la sesión de trabajo se rompe. Cualquier modificación realizada en este estado no se reflejará en la base de datos a menos que el objeto sea reincorporado (merge) a una nueva sesión.
- **Transición:** Se dispara al cerrar la sesión (`session.close()`), limpiar el contexto (`session.clear()`), expulsar un objeto específico (`session.detach()` o `evict()`), o simplemente al finalizar la transacción.

Removed

Es un estado transitorio donde el objeto está destinado a desaparecer físicamente del almacenamiento.

- **Descripción:** La entidad todavía tiene identidad y está asociada a la sesión, pero ha sido marcada para ser borrada de la base de datos en cuanto se complete la unidad de trabajo (unidad atómica).
- **Transición:** Se activa al invocar `session.remove()` o `session.delete()`. También puede dispararse por una operación en cascada (**`CascadeType.REMOVE`**) o por el mecanismo de **`orphanRemoval = true`** al desvincular un objeto de una composición.

Mapeo de Entidades

Anotaciones de JPA y de Hibernate, y otros requisitos

Anotación	Descripción	¿Obligatoria?	JPA/Hibernate
@Entity	Define que una clase es una entidad persistente y que sus instancias deben ser almacenadas en la base de datos.	Sí	JPA
@Id	Define el atributo que actúa como identificador único o clave primaria (PK) de la clase.	Sí	JPA
@GeneratedValue	Indica que el valor del ID se generará automáticamente según una estrategia (ej. <code>IDENTITY</code> , <code>SEQUENCE</code> o <code>TABLE</code>).	No	JPA
@Table	Permite especificar detalles de la tabla, como su nombre (opcional si coincide con la clase) y restricciones de unicidad.	No	JPA
@Column	Define propiedades de la columna mapeada, como su nombre, si permite nulos (<code>nullable</code>) o su longitud máxima.	No	JPA
@Transient	Indica que un atributo debe ser ignorado por el motor de persistencia y no guardarse en la base de datos.	No	JPA
@ManyToOne	Define una relación de muchos a uno. Generalmente es el lado "dueño" que contiene la clave foránea en la tabla.	No	JPA
@OneToMany	Define una relación de uno a muchos. Se usa frecuentemente con <code>mappedBy</code> para indicar que es el lado inverso de la relación.	No	JPA
@ManyToMany	Define una relación de muchos a muchos, lo que implica la creación de una tabla intermedia de unión.	No	JPA
@OneToOne	Define una relación de uno a uno entre dos entidades (ej. entre	No	JPA

Anotación	Descripción	¿Obligatoria?	JPA/Hibernate
	Purchase y Review).		
@JoinColumn	Especifica el nombre físico de la columna que actúa como clave foránea (FK) en la tabla.	No	JPA
@JoinTable	Define los detalles de la tabla intermedia en una relación muchos a muchos, incluyendo los nombres de las columnas de unión.	No	JPA
@Inheritance	Establece la estrategia para mapear una jerarquía de clases (SINGLE_TABLE , JOINED , TABLE_PER_CLASS).	No	JPA
@DiscriminatorColumn	Define la columna usada en la estrategia SINGLE_TABLE para identificar el tipo de clase de cada fila.	No	JPA
@DiscriminatorValue	Indica el valor específico que tendrá la columna discriminadora para una subclase concreta.	No	JPA
@Version	Se utiliza para implementar el control de concurrencia optimista, permitiendo detectar si un objeto fue modificado por otro proceso.	No	JPA
@SQLDelete	Permite sobrescribir el comando DELETE por un UPDATE para implementar el borrado lógico (soft delete).	No	Hibernate
@Where	Define un filtro SQL global que Hibernate aplica automáticamente a todas las consultas de la entidad (ej. para ocultar registros borrados).	No	Hibernate

La clase a ser Mapeada también debe cumplir algunas **reglas estructurales de los POJOs**:

- **Constructor sin argumentos:** La clase debe tener obligatoriamente un constructor por defecto (sin parámetros) para que el framework pueda instanciarla al recuperar datos,.

- **Campos No Finales:** La clase y sus métodos persistentes no deben ser declarados como `final`.
- **Convención sobre Configuración:** Anotaciones como `@Table` o `@Column` son opcionales; si se omiten, JPA asume por convención que el nombre de la tabla es el de la clase y el de las columnas el de los atributos.
- **Generación de IDs:** Aunque no es estrictamente mínima, suele acompañarse la anotación `@Id` con `@GeneratedValue` para que el motor gestione automáticamente el incremento de la clave primaria.

Parámetro mappedBy

`mappedBy` es el parámetro que le dice a JPA cuál es el **lado dueño** de la relación, es decir, qué atributo en la otra clase es el que tiene la FK física en la base de datos.

- El lado con `@ManyToOne` + `@JoinColumn` → **es el dueño** (tiene la FK en la tabla)
- El lado con `@OneToMany(mappedBy = ...)` → **es el inverso** (solo para navegar en Java, no genera nada en la BD)

Si se omite en ambos lados JPA interpreta que hay **dos relaciones independientes** y crea una **tabla intermedia innecesaria** con los IDs de las tablas involucradas.

Persistencia por Alcance

La **Persistencia por Alcance** es un principio que establece que **todo objeto al que se pueda llegar (navegar) a partir de un objeto que ya es persistente, debe volverse persistente a su vez**.

Si un objeto persistente hace referencia a un objeto que todavía es transitorio (`transient`), pueden ocurrir dos escenarios dependiendo de la configuración:

- **Persistencia Automática (Cascada Activa):** Si la relación está configurada con operaciones en cascada, Hibernate **persistirá automáticamente el objeto referenciado**.
- **Error de Persistencia (Cascada Desactivada):** Por defecto, en JPA **no se propaga ninguna operación**. Si intentas confirmar una transacción donde un objeto persistente apunta a uno transitorio y no hay cascada configurada, el sistema fallará.

Estrategias de Generación de IDs

Característica	IDENTITY	SEQUENCE	TABLE
Mecanismo	Esta estrategia delega la generación del ID totalmente al motor de la base de datos,	Utiliza un objeto de base de datos específico llamado Secuencia para	Crea una tabla dedicada en el esquema de la base de datos para simular una secuencia,

Característica	IDENTITY	SEQUENCE	TABLE
	utilizando columnas de tipo "auto-increment"	obtener los valores numéricos antes de realizar el insert	almacenando el último ID generado en una fila
Generación del ID	Después del <code>INSERT</code>	Antes del <code>INSERT</code>	Antes del <code>INSERT</code>
Soporte de Batching	No (lo desactiva)	Sí	Sí
Portabilidad	Alta (MySQL, SQL Server)	Media (Oracle, PostgreSQL)	Máxima (Universal)
Rendimiento	Medio	Excelente	Bajo
Uso de Bloqueos	A nivel de tabla/página	Optimizado por el motor	Bloqueo físico en fila

Fetch Type: LAZY vs. EAGER

La propiedad `fetch` determina la **estrategia de carga** de los objetos asociados en una relación. Esta decisión impacta directamente en la performance (número de consultas SQL) y en el espacio que ocupa el grafo de objetos en la memoria del servidor.

Comparación

Aspecto	<code>FetchType.EAGER</code>	<code>FetchType.LAZY</code>
Concepto	Carga automáticamente las relaciones junto con la entidad principal.	Carga las relaciones solo cuando se accede a ellas por primera vez.
Ventajas	- Datos disponibles inmediatamente. - Evita consultas adicionales y posibles N+1 queries cuando siempre se necesitan las relaciones. - No genera problemas por acceder a relaciones fuera de una sesión activa.	- Consume menos memoria al cargar solo lo necesario. - Recuperación inicial más rápida. - Mejor escalabilidad para grafos de objetos grandes.
Desventajas	- Mayor consumo de memoria al cargar datos que podrían no utilizarse. - Consultas iniciales más lentas debido a múltiples <code>JOINS</code>.	- Puede provocar <code>LazyInitializationException</code> si la sesión está cerrada. - Puede generar muchas consultas adicionales (N+1 queries) al recorrer relaciones.

Aspecto	FetchType . EAGER	FetchType . LAZY
Impacto en memoria	Alto.	Bajo.
Velocidad de consulta inicial	Más lenta.	Más rápida.
Velocidad de acceso posterior	Muy rápida (datos ya cargados).	Puede requerir consultas adicionales.
Riesgo principal	Sobrecarga de memoria y consultas complejas.	Excepciones de inicialización y exceso de consultas.
Uso recomendado	Cuando la relación siempre será utilizada.	Como estrategia por defecto en la mayoría de los casos.

Valores por Defecto para las Anotaciones

Anotación	Valor por Defecto	Justificación
@OneToMany	LAZY	Al tratarse de colecciones que pueden ser muy grandes, el default razonable es no cargarlas para evitar agotar la memoria
@ManyToOne	EAGER	En el estándar JPA oficial es EAGER , sin embargo, Hibernate suele tratar estas relaciones también como LAZY por defecto para ser conservador con los recursos
@ManyToMany	LAZY	Similar a las colecciones 1..N, se busca evitar la hidratación accidental de grandes partes del grafo de objetos

¿Por qué no configurar todas las relaciones como **EAGER** ?

Problema	Descripción
Efecto Cascada de Hidratación	Cargar una entidad puede desencadenar la carga recursiva de múltiples relaciones y subrelaciones.
Agotamiento de Memoria	La aplicación puede terminar cargando una gran porción de la base de datos en memoria, provocando errores <code>OutOfMemory</code> .
Explosión de JOINS	El ORM genera consultas SQL con demasiados <code>JOINS</code> , degradando significativamente el rendimiento de la base de datos.
Pérdida de Control	El desarrollador no puede optimizar consultas específicas, ya que la carga EAGER es fija y siempre se ejecuta.

Reglas Prácticas

Situación	Estrategia recomendada
Relación necesaria en prácticamente todos los casos de uso	EAGER
Relación opcional o potencialmente grande	LAZY
Diseño general de entidades	Preferir LAZY por defecto
Necesidad puntual de traer relaciones	Utilizar JOIN FETCH , EntityGraph o consultas específicas en lugar de marcar la relación como EAGER permanentemente

Operaciones en Cascada

CascadeType	Operación que propaga	Cuándo se activa	Efecto
PERSIST	Creación (persist)	Al persistir la entidad padre	Las entidades hijas transitorias también se guardan automáticamente.
MERGE	Unión (merge)	Al fusionar una entidad detached	Los cambios de los objetos relacionados también se sincronizan con la base de datos.
REMOVE	Eliminación (remove / delete)	Al eliminar la entidad padre	Se eliminan físicamente las entidades relacionadas.
REFRESH	Recarga (refresh)	Al refrescar la entidad padre	Las entidades relacionadas se vuelven a cargar desde la base de datos.
DETACH	Desasociación (detach / evict)	Al expulsar la entidad del Persistence Context	Las entidades relacionadas pasan también a estado detached.
ALL	Todas las anteriores	Según la operación ejecutada	Incluye PERSIST , MERGE , REMOVE , REFRESH y DETACH .

Comparación: `CascadeType.REMOVE` vs `orphanRemoval = true`

Aspecto	<code>CascadeType.REMOVE</code>	<code>orphanRemoval = true</code>
Propósito	Propagar la eliminación del padre a los hijos.	Eliminar hijos que pierden su referencia al padre.
Disparador principal	<code>session.delete(padre)</code> o <code>remove(padre)</code> .	Eliminar el padre o quitar un hijo de la colección.
Eliminación al borrar el padre	Sí.	Sí.
Eliminación al remover un hijo de la colección	No.	Sí.
Manejo de huérfanos	No elimina objetos huérfanos.	Elimina automáticamente objetos huérfanos.
Orientado a	Relaciones donde interesa la propagación del borrado.	Relaciones de composición fuerte.
Nivel de agresividad	Menor.	Mayor.

Mapeos de Herencia

Aspecto	<code>SINGLE_TABLE</code>	<code>JOINED</code>	<code>TABLE_PER_CLASS</code>
Tablas creadas	Una sola tabla para toda la jerarquía.	Una para la superclase y una por cada subclase.	Una tabla por cada clase concreta.
Columna discriminadora	Sí (obligatoria) para identificar el tipo de objeto.	Opcional (Hibernate puede deducirlo por el JOIN).	No (el tipo está implícito en la tabla).
Atributos NULL en BD	Sí: las columnas de subclases deben ser <code>nullable</code> .	No: cada tabla tiene solo sus datos específicos.	No: las tablas son independientes y completas.
Consulta polimórfica	Muy eficiente: Simple <code>SELECT *</code> sobre la tabla.	Ineficiente: Requiere múltiples <code>OUTER JOINS</code> con las hijas.	Muy ineficiente: Requiere un <code>UNION ALL</code> de todas las tablas.
Integridad Referencial	Limitada: No se pueden definir restricciones <code>NOT NULL</code> en atributos de subclase.	Alta: Permite definir restricciones <code>NOT NULL</code> y FK reales.	Alta: Cada tabla soporta sus propias restricciones nativas.
Rendimiento Lectura	Excelente: Ideal para consultas frecuentes	Bajo: El costo de los JOINS permanentes	Bueno para lecturas de una clase concreta,

Aspecto	SINGLE_TABLE	JOINED	TABLE_PER_CLASS
	sobre la superclase.	penaliza la velocidad.	malo para polimórficas.
Nueva subclase	Se agregan nuevas columnas a la tabla única.	Se crea una nueva tabla sin tocar las existentes.	Compleja: Estructura repetida y afecta a las consultas UNION .

Uso Recomendado

Estrategia	Uso
SINGLE_TABLE	Es la opción por defecto y la más performante cuando las subclases tienen pocos atributos propios y se realizan muchas consultas polimórficas
JOINED	Es la más robusta ante cambios futuros y preferible cuando se requiere una normalización estricta e integridad de datos a nivel de base de datos
TABLE_PER_CLASS	Útil solo si casi nunca se realizan consultas polimórficas sobre la superclase, ya que la operación UNION es la más costosa en SQL

Patrón de acceso a datos: DAO y Repository

Característica	Patrón DAO (Data Access Object)	Patrón Repository
Objetivo Principal	Encapsular la lógica de acceso a datos de una tabla o entidad específica.	Simular una colección de objetos en memoria , ocultando la persistencia real.
Enfoque (Foco)	Centrado en los datos y el almacenamiento físico (tablas/filas).	Centrado en el dominio y el comportamiento de una colección clásica.
Nivel de Abstracción	Adaptador hacia la base de datos (abstracción del almacenamiento).	Capa intermedia que representa objetos de negocio (abstracción del dominio).
Protocolo CRUD	Implementa explícitamente todas las operaciones: C reate, R ead, U ppdate, D eleate.	Se concentra en la Recuperación (R) o búsqueda eficiente (ej. <code>findByName</code>).
Uso con ORM	Suele desaprovechar capacidades del ORM al forzar actualizaciones manuales.	Aprovecha la persistencia por alcance ; la creación y actualización son transparentes.
Implementación	Mapeo casi directo a las tablas de la base de datos.	Actúa como una "máscara" que oculta el envío de consultas

Característica	Patrón DAO (Data Access Object)	Patrón Repository (HQL/JPQL).
Crítica Principal	Tiende a generar diseños "débiles" u orientados a procedimientos (modelos anémicos).	Puede complicarse con el tipado estático y volverse repetitivo en su implementación.

Diferencia Conceptual Clave

La distinción más importante reside en el **modelo mental** del desarrollador: mientras que al usar un **DAO** se piensa en términos de manipular registros en tablas, al usar un **Repository** se interactúa con el sistema como si todos los objetos de negocio estuvieran disponibles en la memoria de la aplicación, delegando al repositorio la tarea de recuperarlos eficientemente cuando sea necesario.

Transacciones

Una **transacción** se define como la delimitación de una **unidad de trabajo**. Es un conjunto de operaciones (mensajes enviados a los objetos) que deben ejecutarse bajo las propiedades **ACID**: Atómica, Consistente, Aislada y Duradera.

- En Hibernate, la transacción actúa como un objeto al que se le envían mensajes como `begin()`, `commit()` y `rollback()` para controlar el flujo de persistencia.

La transacción debe gestionarse en la capa de servicio, no en el repositorio. Justificación:

- *Razón 1 — La unidad de trabajo es de negocio, no de acceso a datos*
- *Razón 2 — El repositorio debe ser reutilizable y simple*

Consultas con HQL y JPQL, y comparación con SQL

Aspecto	SQL Nativo	HQL / JPQL
Enfoque	Centrado en el modelo relacional (tablas y columnas).	Centrado en el modelo de objetos (clases y sus atributos).
Sujeto de Consulta	Las tablas y columnas físicas definidas en el esquema de la base de datos.	Las clases de entidad y los atributos definidos en el código Java.
Resultado	Un conjunto de tuplas o filas de datos planos .	Una colección de objetos gestionados (<i>Managed</i>) que mantienen su identidad y estado.
Herencia y Polimorfismo	El motor es "ignorante" de la herencia; requiere <code>JOINS</code> o	Entiende la jerarquía; consultar una superclase (ej. <code>FROM User</code>) incluye

Aspecto	SQL Nativo	HQL / JPQL
	<code>UNIONS</code> manuales según la tabla.	automáticamente a todas las subclases.
Navegación (Asociaciones)	Requiere especificar cláusulas <code>JOIN</code> y condiciones de igualdad entre claves explícitamente.	Utiliza expresiones de ruta (<i>Path Expressions</i>) con el operador punto (ej. <code>p.user.name</code>).
Colecciones	No las reconoce como contenedores; son registros en otras tablas vinculados por una clave.	Comprende que un atributo puede ser un <code>Set</code> o <code>List</code> , permitiendo hacer <code>JOIN</code> directo sobre el atributo.
Tipos de Datos	Atado estrictamente a los tipos del motor físico (ej. <code>VARCHAR2</code> , <code>TIMESTAMP</code>).	Opera sobre tipos de Java (<code>Enum</code> , <code>Date</code> , <code>String</code>) y conoce el mapeo al motor subyacente.
Portabilidad	Atado estrictamente al motor de base de datos específico utilizado.	Alta portabilidad; Hibernate traduce la consulta al dialecto SQL del motor configurado.

Consejos y Sintaxis Clave

Gestión de Parámetros

- **Preferencia por Parámetros Nombrados:** En lugar de usar índices posicionales (`?1`), se recomienda usar parámetros nombrados (`:nombreParametro`) junto con la anotación `@Param` . Esto hace que la consulta sea más legible y robusta ante cambios en la firma del método.
- **Seguridad:** El uso de parámetros (ya sean nombrados o posicionales) es vital para prevenir ataques de inyección SQL, ya que el framework se encarga del escape de los valores.

Optimización de la Carga (Fetching)

- **Uso de JOIN FETCH:** Cuando se necesita acceder a una relación marcada como `LAZY` fuera del contexto de una sesión activa (evitando la `LazyInitializationException`), se debe usar `JOIN FETCH` en la consulta. Esto obliga a Hibernate a traer la entidad relacionada en la misma consulta SQL.
- **Consultas Polimórficas:** Recordar que si la jerarquía usa `SINGLE_TABLE` , las consultas polimórficas son muy rápidas (un solo `SELECT`), mientras que en `TABLE_PER_CLASS` implican un costoso `UNION ALL` .

Funciones de Agregación y Operadores

- **Funciones Estándar:** Se pueden usar funciones como `COUNT`, `SUM`, `AVG`, `MIN` y `MAX` directamente sobre los atributos de los objetos.
- **Operador SIZE:** Es extremadamente útil en HQL para obtener la cantidad de elementos en una colección sin tener que cargar la colección completa en memoria (ej. `SELECT SIZE(r.stops) FROM Route r`).
- **Palabra clave DISTINCT:** Siempre usarla cuando se realicen `JOINS` sobre colecciones para evitar filas duplicadas en el resultado de objetos (ej. `SELECT DISTINCT p.user FROM Purchase p`).

Consultas con DTOs

- **Constructor de DTO en Query:** Si no se desea recuperar toda la entidad, se puede proyectar el resultado directamente a un DTO usando la sintaxis `SELECT new com.paquete.MiDTO(p.id, p.name) FROM ...`. Esto mejora la performance al reducir el tráfico de datos y evitar ciclos de serialización.

¿Cuándo usar SQL Nativo (`nativeQuery = true`)?

Solo se debe recurrir a SQL nativo en casos específicos donde HQL/JPQL sea insuficiente:

- **Funciones propietarias:** Cuando se necesitan funciones específicas de un motor (como `DATE_FORMAT` en MySQL) que no están estandarizadas.
- **Operaciones Masivas:** Para `UPDATE` o `DELETE` que afecten a miles de registros, ya que SQL nativo opera directamente en la BD sin cargar cada objeto al *Persistence Context*.
- **Optimización Extrema:** Para forzar el uso de índices específicos o planes de ejecución manuales (hints).

Spring Data JPA: Query Methods vs. `@Query`

- **Query Methods:** Ideales para búsquedas simples por uno o dos atributos (ej. `findByEmail`). Se derivan automáticamente del nombre del método.
- **@Query:** Necesario cuando la consulta requiere `JOINS` complejos, agrupamientos (`GROUP BY`), o funciones que la derivación por nombre no puede interpretar.

Spring Data JPA

Spring Data JPA es una **capa de abstracción superior** que se sitúa por encima del ORM (Hibernate) dentro del ecosistema Spring. Su objetivo es simplificar significativamente la implementación de la capa de acceso a datos (repositorios), permitiendo que el desarrollador se enfoque en definir interfaces en lugar de escribir clases de implementación concretas.

Los problemas que resuelve respecto de usar Hibernate directamente son:

- **Reducción de Código Boilerplate:** En Hibernate directo (Práctica 1), el desarrollador debe crear clases de implementación, inyectar la `SessionFactory`, gestionar sesiones con `getCurrentSession()` y escribir manualmente métodos para cada operación básica (guardar, buscar, borrar). Spring Data JPA elimina esta repetición implementando automáticamente estas operaciones en tiempo de ejecución.
- **Abstracción del Repositorio:** Permite tratar la persistencia como una simple colección de objetos mediante interfaces, desacoplando la lógica de negocio de los detalles técnicos de la `Session` de Hibernate.
- **Generación Automática de Consultas:** Evita tener que escribir HQL/JPQL para búsquedas simples, ya que puede derivar la consulta directamente del nombre del método definido en la interfaz.

Las situaciones donde Spring Data JPA simplifica el código serían:

- **Las operaciones CRUD básicas** ya que Spring provee todas las implementaciones de estas operaciones sin nosotros tener que escribir código.
- **Consultas de búsqueda simples** ya que al usar la técnica de Query Methods, el framework se vuelve capaz de interpretar el nombre del método, generar el HQL correspondiente y ejecutarlo automáticamente.

Responsabilidades

Tarea / Responsabilidad	JDBC	Hibernate (ORM)	Spring Data JPA (Abstracción)	Explicación
Abrir y cerrar conexiones	X			Comunicación básica con la BD.
Ejecutar SQL contra la BD	X			Envía sentencias SQL al motor de base de datos.
Mapear <code>ResultSet</code> a objetos Java		X		Convierte filas en entidades Java.
Leer anotaciones <code>@Entity</code> , <code>@Table</code> , <code>@Column</code>		X		Define cómo se relacionan objetos y tablas.
Gestionar estados de entidades (<code>Transient</code> , <code>Managed</code> , <code>Detached</code> , <code>Removed</code>)		X		Controla el ciclo de vida de las entidades.

Tarea / Responsabilidad	JDBC	Hibernate (ORM)	Spring Data JPA (Abstracción)	Explicación
Mantener el Persistence Context (Cache L1)		X		Evita lecturas innecesarias de la BD.
Dirty Checking		X		Detecta cambios y genera UPDATE automáticamente.
Ejecutar flush() y sincronizar cambios		X		Decide cuándo enviar INSERT , UPDATE y DELETE .
Implementar Lazy Loading		X		Carga relaciones solo cuando se necesitan.
Traducir JPQL/HQL a SQL		X		Genera SQL específico para cada base de datos.
Manejar el pool de conexiones		X		Reutiliza conexiones para mejorar rendimiento.
Crear repositorios automáticamente			X	Genera implementaciones en runtime.
Implementar save() , findById() , deleteById()			X	Evita escribir código CRUD repetitivo.
Derivar consultas desde nombres de métodos (findByName)			X	Genera consultas automáticamente.
Soporte de paginación (Pageable)			X	Permite paginar resultados fácilmente.
Soporte de ordenamiento (Sort)			X	Agrega ordenamiento sin código extra.
Ocultar el uso directo de Session			X	Simplifica el acceso a datos.
Manejar @Transactional			X	Gestiona límites transaccionales.

Configuraciones

Aspecto	Hibernate tradicional	Spring Boot + Spring Data JPA
Lugar de configuración	Clase Java (<code>HibernateConfiguration</code>) o XML.	Archivo <code>application.properties</code> .
Configuración de conexión	Se configura manualmente en código.	Mediante propiedades <code>spring.datasource.*</code> .
Creación de <code>SessionFactory</code>	El desarrollador la crea e inyecta manualmente.	Spring Boot crea automáticamente el <code>EntityManagerFactory</code> .
Obtención de sesiones	Uso explícito de <code>SessionFactory</code> y <code>Session</code> .	Spring Data JPA abstrae el acceso mediante repositorios.
Configuración del dialecto	Se establece manualmente en Hibernate.	<code>spring.jpa.database-platform</code> o <code>hibernate.dialect</code> .
Gestión del esquema	Configuración manual de Hibernate.	<code>spring.jpa.hibernate.ddl-auto</code> .
Visualización de SQL	Configuración específica de Hibernate.	<code>spring.jpa.show-sql=true</code> .
Cantidad de código de infraestructura	Alta.	Muy baja (convención sobre configuración).

Jerarquía de Repositorios de Spring Data JPA

Interfaz	Hereda de	Métodos / Capacidades principales
<code>Repository<T, ID></code>	-	No posee métodos. Permite a Spring identificar repositorios y capturar los tipos genéricos.
<code>CrudRepository<T, ID></code>	<code>Repository<T, ID></code>	Operaciones CRUD básicas. <code>save()</code> , <code>findById()</code> , <code>findAll()</code> , <code>existsById()</code> , <code>count()</code> , <code>delete()</code> ,

Interfaz	Hereda de	Métodos / Capacidades principales
		<code>deleteById()</code> , <code>deleteAll()</code> .
PagingAndSortingRepository<T, ID>	<code>CrudRepository<T, ID></code>	Paginación y ordenamiento. <code>findAll(Pageable pageable)</code> , <code>findAll(Sort sort)</code> .
JpaRepository<T, ID>	<code>PagingAndSortingRepository<T, ID></code>	Funcionalidades específicas de JPA y operaciones masivas. <code>flush()</code> , <code>saveAndFlush()</code> , operaciones batch, retorno de <code>List<T></code> en lugar de <code>Iterable<T></code> .

Proxy Dinámico en Spring Data JPA

Es un objeto generado automáticamente por Spring en tiempo de ejecución (*runtime*) que implementa las interfaces de repositorio sin necesidad de que el desarrollador escriba una clase concreta.

Existe porque las interfaces de repositorio (`UserRepository` , `TourRepository` , etc.) **no contienen código ejecutable propio**. Spring necesita generar una implementación real para que puedan utilizarse.

¿Qué hace Spring al arrancar la aplicación?

- Escanea el proyecto en busca de interfaces que hereden de `Repository` .
- Genera automáticamente una implementación concreta para cada repositorio.
- Registra dicha implementación como un **Bean** dentro del contenedor de Spring.

¿Qué hace el Proxy?

Actúa como **intermediario** entre la **aplicación** y la **capa de persistencia**:

- Intercepta las llamadas a los métodos del repositorio.

- Determina qué lógica debe ejecutarse.
- Delega la ejecución a Spring Data JPA y Hibernate.

Métodos CRUD

Para métodos estándar como:

- `save()`
- `findById()`
- `findAll()`
- `deleteById()`

el **Proxy** delega la ejecución a las implementaciones internas proporcionadas por **Spring Data JPA e Hibernate**.

Query Methods

Cuando se invoca un método el Proxy:

1. Analiza el nombre del método buscando **prefijos** (intención) y **atributos** de la entidad.
2. Deriva el HQL/JPQL correspondiente y lo traduce a SQL nativo según el dialecto de la base de datos.

Transacciones

El Proxy puede envolver la ejecución de los métodos con lógica transaccional, gestionando anotaciones como: `@Transactional`

Gestión de sesiones JPA/Hibernate

El Proxy administra internamente:

- El acceso al `EntityManager`.
- La comunicación con Hibernate.
- La interacción con la base de datos.

Beneficio principal

El desarrollador solo define la interfaz del repositorio; Spring genera automáticamente toda la implementación necesaria en tiempo de ejecución.

QueryMethods y @Query

Los QueryMethods permiten que el framework genere automáticamente la implementación de las consultas analizando simplemente el **nombre del método** definido en la interfaz del repositorio.

Limitaciones de los QueryMethods

- **Complejidad y Legibilidad:** A medida que se agregan más condiciones (usando `And` , `Or`), el nombre del método se vuelve excesivamente largo y difícil de leer, lo que compromete el mantenimiento del código.
- **Capacidad de Expresión Limitada:** No permiten realizar operaciones de **agregación complejas** (como `GROUP BY` o `HAVING`), ni manejar **subconsultas** de forma nativa en el nombre del método.
- **JOINS No Explícitos:** Aunque permiten navegar por asociaciones simples (ej. `findByUserUsername`), los Query Methods resultan insuficientes para **JOINS complejos** que involucran múltiples niveles o condiciones específicas de unión que no se deducen de la jerarquía de propiedades.
- **Naturaleza Estática:** Están definidos en tiempo de compilación por el nombre; no permiten construir criterios de búsqueda de forma dinámica según parámetros que pueden o no estar presentes en tiempo de ejecución.

Sintaxis y Palabras Clave para Query Methods

La estructura básica es: `[Prefijo][Atributo][PalabraClave][OperadorLógico][Atributo]...`

Categoría	Palabras clave	Función	Ejemplo
Prefijos de intención	<code>findBy</code> , <code>readBy</code> , <code>getBy</code> , <code>queryBy</code>	Realizan consultas de selección (SELECT).	<code>findByEmail()</code>
	<code>existsBy</code>	Verifica si existe un registro.	<code>existsByUsername()</code>
	<code>countBy</code>	Cuenta registros que cumplen una condición.	<code>countByStatus()</code>
	<code>deleteBy</code> , <code>removeBy</code>	Eliminan registros.	<code>deleteByEmail()</code>
Operadores lógicos	<code>And</code>	Ambas condiciones deben cumplirse.	<code>findByNameAndPrice()</code>
	<code>Or</code>	Al menos una condición debe	<code>findByEmailOrUsername()</code>

Categoría	Palabras clave	Función	Ejemplo
		cumplirse.	
Comparación	(por defecto), Is , Equals	Igualdad.	findByName()
	Not	Desigualdad.	findByStatusNot()
	Between	Valor dentro de un rango.	findByPriceBetween()
	LessThan , LessThanEqual	Menor que / menor o igual.	findByPriceLessThan()
	GreaterThan , GreaterThanEqual	Mayor que / mayor o igual.	findByAgeGreaterThanEqual()
	IsNull , IsNotNull (NotNull)	Verifica nulidad.	findByDescriptionIsNull()
Strings	Like	Búsqueda por patrón (% manual).	findByNameLike()
	NotLike	Exclusión por patrón.	findByNameNotLike()
	StartingWith	Comienza con un texto.	findByNameStartingWith()
	EndingWith	Termina con un texto.	findByNameEndingWith()
	Containing	Contiene un texto en cualquier posición.	findByNameContaining()
Ordenamiento	OrderBy	Ordena resultados por un atributo.	findAllByOrderByPriceAsc()
	Asc	Orden ascendente.	OrderByPriceAsc
	Desc	Orden descendente.	OrderByPriceDesc
Limitación	First	Devuelve el primer resultado o los primeros N.	findFirstByOrderByPriceDesc()
	Top	Limita la cantidad de resultados.	findTop3ByOrderByPriceDesc()

Gestión de Parámetros para QueryMethods

La asociación es **estrictamente posicional**.

- Si el método es `findByNameAndPrice(String n, Double p)`, el framework asocia el primer atributo del nombre (`Name`) con el primer parámetro (`n`), y así sucesivamente.

Gestión de Parámetros para @Query

Aspecto	Posicionales (?1 , ?2)	Nombrados (@Param)
Identificación	Por posición.	Por nombre.
Legibilidad	Baja.	Alta.
Mantenimiento	Más difícil.	Más sencillo.
Sensibilidad al orden	Alta.	Ninguna.
Refactorización	Riesgosa.	Segura.
Recomendación	Solo consultas simples.	Forma recomendada.

Ejemplos

Posicionales

```
@Query("SELECT u FROM User u WHERE u.email = ?1 AND u.name = ?2")
User findUser(String email, String name);
```

Nombrados

```
@Query("""
SELECT u
FROM User u
WHERE u.email = :email
      AND u.name = :name
""")
User findUser(
    @Param("email") String email,
    @Param("name") String name
);
```

Paginación y Ordenamiento Dinámico

Para evitar nombres de métodos extremadamente largos, se pueden pasar parámetros especiales:

- **Pageable**: Permite definir el número de página, tamaño y ordenamiento de forma dinámica desde el servicio.
- **Tipo de retorno**:
 - **Page<T>**: Ejecuta un `COUNT(*)` adicional para saber el total de páginas. Ideal para navegadores de páginas clásicos.
 - **Slice<T>**: No ejecuta el conteo; solo sabe si hay una página siguiente. Ideal para "scroll infinito".

HQL vs JPQL

Aspecto	JPQL	HQL
Qué es	Lenguaje estándar definido por JPA.	Lenguaje propio de Hibernate.
Portabilidad	Funciona con cualquier implementación JPA.	Depende de Hibernate.
Capacidades	Solo las definidas por el estándar JPA.	Es un superconjunto de JPQL (incluye funcionalidades extra).
Compatibilidad	Toda consulta JPQL es válida en Hibernate.	Algunas consultas HQL no son válidas en otros proveedores JPA.
Uso en <code>@Query</code>	Es el lenguaje asumido por defecto.	Solo las partes compatibles con JPQL.

Cuándo usar QueryMethod y cuando usar @Query

Escenario	Estrategia Recomendada	Justificación
Consultas simples por 1 o 2 atributos	Query Method	Es rápido, no requiere escribir HQL y es muy legible.
Consultas con <code>GROUP BY</code> , <code>HAVING</code> o subconsultas	@Query (JPQL)	Los Query Methods no soportan estas operaciones de agregación.
Consultas con JOINS muy complejos o múltiples niveles	@Query (JPQL)	El nombre del método se vuelve inmanejable y difícil de mantener.
Funciones específicas del motor (ej: <code>DATE_FORMAT</code>)	@Query(nativeQuery = true)	JPQL es estándar; para funciones propietarias se requiere SQL nativo.

Profundidad en @Transactional

Si usamos el parámetro `readOnly = true`, indicamos que transacción es **solo de lectura**, es decir, no se esperan modificaciones en la base de datos.

Se usa sólo para **métodos de consulta**, en otros casos no incluimos el parámetro.

Optimizaciones

Optimización	Beneficio
Desactiva Dirty Checking	Hibernate deja de monitorear cambios en las entidades.
Evita Flush Automático	No intenta ejecutar <code>INSERT</code> , <code>UPDATE</code> ni <code>DELETE</code> .
Reduce uso de memoria	No mantiene snapshots de las entidades cargadas.
Optimiza lecturas en la BD	Algunos motores y drivers utilizan transacciones más livianas.

Rollback Automático

Si ocurre una excepción durante una transacción, Spring puede **deshacer todos los cambios realizados** para mantener la consistencia de la base de datos.

Tipo de excepción	¿Hace rollback?
<code>RuntimeException</code>	Sí
<code>Error</code>	Sí
<code>Exception</code> (checked)	No
Excepciones propias que heredan de <code>Exception</code>	No

DTOs

Un **DTO (Data Transfer Object)** es un objeto diseñado exclusivamente para **transportar datos** entre diferentes procesos o capas de una aplicación. Su fin primordial es reducir el número de llamadas a métodos entre procesos. Se distingue de las entidades en que es un objeto "plano" que **no contiene lógica de negocio**, limitándose únicamente a poseer atributos y sus correspondientes métodos *getters* y *setters*.

Por qué no devolver entidades JPA de una

No devolvemos las entidades de una por varias razones:

- **Restricciones transaccionales:** No es posible leer información de una entidad fuera del contexto de una **transacción activa**.
- **Seguridad y Encapsulamiento:** Las entidades contienen **lógica de negocio** y un protocolo público que el cliente o la interfaz de usuario no deberían poder invocar indebidamente.
- **Acoplamiento:** Exponer entidades acopla fuertemente la interfaz con el modelo de datos.
- **Riesgos de Serialización:** Las entidades suelen tener relaciones bidireccionales que pueden generar **ciclos de serialización** infinitos al intentar convertirlas a formatos como JSON para un endpoint.
- etc.

Borrado Lógico

El **borrado lógico (soft delete)** es una estrategia de persistencia que consiste en evitar la eliminación física de un registro de la base de datos, optando en su lugar por marcarlo con un estado de **inactivo o borrado**. Generalmente, esto se logra mediante un campo booleano (ej. `deleted = true`) o un campo de fecha (`deletedAt`), permitiendo que el dato permanezca en el soporte físico aunque sea invisible para la lógica ordinaria de la aplicación.

Estrategia de Campo Booleano (`active` / `deleted`)

Es la forma más simple y común de implementación.

- **Mecanismo:** Se agrega un atributo booleano a la entidad (por ejemplo, `private boolean deleted = false`).
- **Implementación:** Se utiliza la anotación `@SQLDelete` para que, al invocar el método `delete()`, Hibernate ejecute un `UPDATE` cambiando el valor del booleano a `true` en lugar de un `DELETE` físico.
- **Filtrado:** Se combina con `@Where(clause = "deleted = false")` para que todas las consultas ordinarias (`findAll`, `findById`) ignoren automáticamente los registros marcados como borrados.

Estrategia de Campo de Fecha (`deletedAt`)

Esta estrategia es más avanzada y orientada a la auditoría.

- **Mecanismo:** Se utiliza un campo de tipo fecha/tiempo (ej. `LocalDateTime deletedAt`). Si el objeto está activo, el campo es `null`; si está borrado, contiene la fecha exacta de la eliminación.
- **Implementación:** El `@SQLDelete` realiza un `UPDATE` asignando la fecha actual al campo `deleted_at`.
- **Filtrado:** La cláusula global sería `@Where(clause = "deleted_at IS NULL")`.

Bases de Datos NoSQL y Relaciones

Una **Base de Datos NoSQL** es una categoría de sistemas de gestión de datos **no relacionales**, **sin esquema fijo** (schemaless) y diseñados específicamente para **escalar horizontalmente** en entornos distribuidos o clusters.

Equivalencias RDBMS ↔ MongoDB

RDBMS	MongoDB	Diferencia clave
Base de Datos	Base de Datos	Concepto equivalente.
Tabla / Relación	Colección (Collection)	Una colección no requiere esquema fijo.
Fila / Tupla	Documento (Document)	Un documento puede contener estructuras anidadas.
Columna	Campo / Atributo (Field)	No existe una definición global de columnas.

Claves Foráneas

MongoDB no tiene **Foreign Keys reales**. Las relaciones se modelan mediante referencias (`_id`) o documentos embebidos.

Aspecto	RDBMS	MongoDB
Claves foráneas	Sí	No
Integridad referencial automática	Sí	No
Relaciones	FK + JOIN	Referencias o Embedding
Control de consistencia	Base de datos	Aplicación
JOINS	Nativos	<code>\$lookup</code>

Índices

Tipo	Uso
<code>_id</code> (automático)	Identificador único.
Single Field	Un atributo del documento.
Compound	Varios atributos del documento.

Tipo	Uso
Geospatial	Ubicación geográfica.
Embedded Field	Campos dentro de subdocumentos.

Vistas

Aspecto	Vista Estándar	Vista Materializada
Descripción	Son similares a las vistas de un RDBMS tradicional. Actúan como una consulta predefinida que se ejecuta en el momento en que se solicita la información.	Son vistas cuyos resultados se han precalculado y guardado físicamente en una colección de la base de datos.
Almacena datos	No	Sí
Datos actualizados	Siempre	Puede quedar desactualizada
Rendimiento de lectura	Menor	Mayor
Espacio en disco	No ocupa	Sí ocupa
Similar a	VIEW SQL	Materialized View SQL

Validación de Documentos

Método	Descripción
Validación de esquema	MongoDB puede imponer restricciones opcionales.
Validación en aplicación	La aplicación verifica estructura y tipos.
Capa de traducción	Permite evolucionar versiones de documentos.

Transacciones MongoDB vs RDBMS

Aspecto	RDBMS	MongoDB
Unidad transaccional	Varias tablas y filas	Documento
Atomicidad	Global	Documento
JOINS	Frecuentes	Se intenta evitarlos

Aspecto	RDBMS	MongoDB
Modelo ideal	Normalización	Agregados embebidos

Relaciones entre documentos

Característica	Documentos Embebidos (Embedding)	Referencias (Referencing)
Concepto	Captura datos relacionados dentro de una estructura jerárquica en un único documento .	Mantiene documentos separados y establece vínculos mediante el identificador _id .
Implementación	Los objetos hijos se guardan como subdocumentos o vectores (arrays) dentro de un atributo del documento padre.	Se almacena el _id del documento relacionado como un atributo. Requiere consultas adicionales o el uso de \$lookup para recuperar la información completa.
Ventajas	• Rendimiento de lectura: Recupera toda la información en una sola operación, evitando saltos entre colecciones. • Atomicidad: Las operaciones de escritura sobre el documento son atómicas (indivisibles). • Localidad de datos: Ideal para clusters, asegurando que los datos relacionados residan en el mismo nodo.	• Normalización: Evita la duplicación y facilita la actualización de datos compartidos. • Relaciones masivas: Adecuado para escalas grandes (ej. "uno a millones") donde el anidamiento rompería el límite de tamaño. • Flexibilidad: Permite consultar cada entidad de forma independiente.
Desventajas	• Límite de tamaño: Los documentos tienen un máximo físico de 16 MB. • Redundancia: Datos repetidos complican actualizaciones y pueden generar inconsistencias. • Acceso independiente: Es costoso consultar elementos hijos de forma aislada sin procesar al padre.	• Consultas múltiples: Requiere varios llamados a la base de datos, lo que aumenta la latencia. • Sin integridad automática: El motor de MongoDB es "ignorante" de la relación lógica; la integridad es responsabilidad de la aplicación.
Casos de Uso Recomendados	• Relaciones de tipo Uno a pocos.	• Relaciones de tipo Uno a Muchos masivas.

Característica	Documentos Embebidos (Embedding)	Referencias (Referencing)
	• Datos que Siempre se consultan juntos. • Relación de composición. • Necesito atomicidad.	• Relaciones Muchos a Muchos. • Entidades independientes. • Información compartida. • Evitar redundancia.

Comandos más usados

Operación	Descripción	Ejemplo
Seleccionar/Crear BD	Cambia a una base de datos. Si no existe, se crea al insertar datos.	<code>use tours</code>
Crear colección	Crea una colección explícitamente.	<code>db.createCollection("recorridos")</code>
Insertar un documento	Inserta un único documento.	<code>db.recorridos.insertOne({ nombre: "City Tour", precio: 200 })</code>
Insertar varios documentos	Inserta múltiples documentos en una sola operación.	<code>db.recorridos.insertMany([{ nombre: "Tour A" }, { nombre: "Tour B" }])</code>
Consultar todos los documentos	Recupera todos los documentos de la colección.	<code>db.recorridos.find()</code>
Consultar con formato legible	Muestra los documentos indentados para facilitar la lectura.	<code>db.recorridos.find().pretty()</code>
Filtrar por igualdad	Recupera documentos que cumplen una condición exacta.	<code>db.recorridos.find({ nombre: "Museum Tour" })</code>
Mayor que (\$gt)	Recupera documentos con valores mayores al indicado.	<code>db.recorridos.find({ precio: { \$gt: 60000 } })</code>
Menor que (\$lt)	Recupera documentos con valores menores al indicado.	<code>db.recorridos.find({ precio: { \$lt: 50000 } })</code>
Mayor o igual (\$gte)	Recupera documentos con valores mayores o iguales al indicado.	<code>db.recorridos.find({ precio: { \$gte: 50000 } })</code>

Operación	Descripción	Ejemplo
Menor o igual (<code>\$lte</code>)	Recupera documentos con valores menores o iguales al indicado.	<code>db.recorridos.find({ precio: { \$lte: 50000 } })</code>
Distinto (<code>\$ne</code>)	Recupera documentos cuyo valor sea diferente al indicado.	<code>db.recorridos.find({ nombre: { \$ne: "Museum Tour" } })</code>
AND implícito	Combina varias condiciones que deben cumplirse simultáneamente.	<code>db.recorridos.find({ precio: { \$gt: 50000 }, totalKm: { \$gt: 10 } })</code>
OR (<code>\$or</code>)	Recupera documentos que cumplen al menos una condición.	<code>db.recorridos.find({ \$or: [{ precio: 30000 }, { precio: 50000 }] })</code>
Buscar valor en array	Busca documentos donde un array contiene un valor.	<code>db.recorridos.find({ stops: "San Telmo" })</code>
Buscar múltiples valores en array (<code>\$all</code>)	Verifica que todos los elementos existan en el array.	<code>db.recorridos.find({ stops: { \$all: ["San Telmo", "Recoleta", "Avenida de Mayo"] } })</code>
Buscar por patrón (<code>\$regex</code>)	Búsqueda parcial de texto.	<code>db.recorridos.find({ stops: { \$regex: "Museo" } })</code>
Tamaño de array (<code>\$size</code>)	Busca arrays con tamaño exacto.	<code>db.recorridos.find({ stops: { \$size: 5 } })</code>
Existencia de campo (<code>\$exists</code>)	Verifica si un campo existe o no.	<code>db.recorridos.find({ totalKm: { \$exists: false } })</code>
Proyección	Devuelve solo determinados campos.	<code>db.recorridos.find({}, { nombre: 1, precio: 1, _id: 0 })</code>
Actualizar un documento	Modifica el primer documento encontrado.	<code>db.recorridos.updateOne({ nombre: "Cultural Odyssey" }, { \$set: { totalKm: 12 } })</code>
Actualizar varios documentos	Modifica todos los documentos encontrados.	<code>db.recorridos.updateMany({}, { \$set: { activo: true } })</code>
Modificar un campo (<code>\$set</code>)	Crea o actualiza un atributo.	<code>db.recorridos.updateOne({ nombre: "Tour" }, { \$set: { precio: 500 } })</code>
Incrementar valor (<code>\$inc</code>)	Incrementa un número.	<code>db.recorridos.updateMany({}, { \$inc: { precio: 5000 } })</code>
Multiplicar valor (<code>\$mul</code>)	Multiplica un valor numérico.	<code>db.recorridos.updateMany({}, { \$mul: { precio: 1.1 } })</code>

Operación	Descripción	Ejemplo
Agregar elemento a array (<code>\$push</code>)	Agrega un elemento al final de un array.	<code>db.recorridos.updateOne({ nombre: "Delta Tour" }, { \$push: { stops: "Tigre" } })</code>
Crear array y agregar elemento	Crea el array si no existe y agrega un valor.	<code>db.recorridos.updateOne({ nombre: "Urban Exploration" }, { \$push: { tags: "Gastronomia" } })</code>
Eliminar elemento de array (<code>\$pull</code>)	Elimina elementos que coincidan.	<code>db.recorridos.updateOne({ nombre: "Tour" }, { \$pull: { stops: "Recoleta" } })</code>
Eliminar un documento	Elimina el primer documento encontrado.	<code>db.recorridos.deleteOne({ nombre: "Temporal Route" })</code>
Eliminar varios documentos	Elimina todos los documentos encontrados.	<code>db.recorridos.deleteMany({ precio: { \$lt: 30000 } })</code>
Ordenar ascendente	Ordena de menor a mayor.	<code>db.recorridos.find().sort({ precio: 1 })</code>
Ordenar descendente	Ordena de mayor a menor.	<code>db.recorridos.find().sort({ precio: -1 })</code>
Limitar resultados	Devuelve una cantidad específica de documentos.	<code>db.recorridos.find().limit(5)</code>
Contar documentos	Cuenta todos los documentos de la colección.	<code>db.recorridos.countDocuments()</code>

Algún tip para ejercicios

Si el enunciado dice por ejemplo...	Pensar en...
"precio mayor a"	<code>\$gt</code>
"precio menor a"	<code>\$lt</code>
"mayor o igual"	<code>\$gte</code>
"menor o igual"	<code>\$lte</code>
"distinto de"	<code>\$ne</code>
"contiene stop X"	<code>stops: "X"</code>
"contiene X e Y e Z"	<code>\$all</code>
"no contiene X"	<code>\$ne</code>

Si el enunciado dice por ejemplo...	Pensar en...
"campo no definido"	<code>\$exists: false</code>
"mostrar solo algunos campos"	Proyección
"agregar elemento a lista"	<code>\$push</code>
"quitar elemento de lista"	<code>\$pull</code>
"aumentar un porcentaje"	<code>\$mul</code>
"aumentar una cantidad fija"	<code>\$inc</code>
"actualizar campo"	<code>\$set</code>
"eliminar documento"	<code>deleteOne()</code>
"cantidad de documentos"	<code>countDocuments()</code>

Aggregation Framework

El **Aggregation Framework** es una herramienta de MongoDB diseñada para realizar procesamiento de datos complejo, agrupamiento y análisis en tiempo real.

Características Principales

- **Funcionamiento por Etapas:** La consulta se organiza como una serie de **etapas secuenciales**.
- **Propósito:** Permite superar las limitaciones del lenguaje de consulta estándar (`find()`) al manejar lógicas que requieren cálculos, uniones de colecciones o transformaciones profundas de la estructura de los documentos.
- **Etapas (Stages) fundamentales:**
 - *`$match`*
 - *`$group`*
 - *`$project`*
 - *`$lookup`*
 - *`$unwind`*
 - *`$sort` y `$limit`*
- **Salida de datos:** El resultado puede ser devuelto a la aplicación o guardado físicamente en una nueva colección utilizando la etapa **`$out`**, lo que permite generar vistas materializadas.

Información de las etapas

Etapa	¿Para qué sirve?	Ejemplo
<code>\$match</code>	Filtrar documentos (equivalente a WHERE)	<code>{ \$match: { precio: { \$gte: 90000 } } }</code>
<code>\$project</code>	Elegir o calcular campos	<code>{ \$project: { nombre: 1, precio: 1 } }</code>
<code>\$lookup</code>	Hacer JOIN entre colecciones	Ver stops de una ruta
<code>\$unwind</code>	Desarmar arrays en documentos individuales	Procesar cada stop por separado
<code>\$group</code>	Agrupar y calcular estadísticas	Promedios, conteos, máximos
<code>\$sort</code>	Ordenar resultados	<code>{ \$sort: { precio: -1 } }</code>
<code>\$limit</code>	Limitar cantidad de resultados	<code>{ \$limit: 5 }</code>
<code>\$sample</code>	Obtener elementos aleatorios	<code>{ \$sample: { size: 5 } }</code>
<code>\$addFields</code>	Agregar campos calculados	Cantidad de stops
<code>\$out</code>	Guardar resultado en una colección nueva	Crear <code>rutas_economicas</code>
<code>\$count</code>	Contar documentos	<code>{ \$count: "total" }</code>

Operadores de agrupación más usados dentro de `$group`

Operador	Función
<code>\$sum</code>	Suma valores
<code>\$avg</code>	Promedio
<code>\$max</code>	Máximo
<code>\$min</code>	Mínimo
<code>\$first</code>	Primer valor
<code>\$last</code>	Último valor
<code>\$push</code>	Construye un array
<code>\$addToSet</code>	Array sin repetidos

Algún tip para ejercicios

Si el enunciado dice por ejemplo...	Pensar en...
"precio mayor a"	<code>\$match</code>

Si el enunciado dice por ejemplo...	Pensar en...
"contar"	<code>\$group + \$sum</code> o <code>\$count</code>
"promedio"	<code>\$group + \$avg</code>
"máximo"	<code>\$group + \$max</code>
"mínimo"	<code>\$group + \$min</code>
"unir colecciones"	<code>\$lookup</code>
"trabajar con arrays"	<code>\$unwind</code>
"ordenar"	<code>\$sort</code>
"primeros N"	<code>\$limit</code>
"aleatorio"	<code>\$sample</code>
"crear colección nueva"	<code>\$out</code>
"agregar campo calculado"	<code>\$addFields</code>

Redis

Redis (Remote Dictionary Server) es una base de datos NoSQL clave-valor en memoria (RAM).

Comparación

Aspecto	Redis	RDBMS (MySQL, PostgreSQL)	MongoDB
Modelo	Clave-Valor	Tablas y relaciones	Documentos JSON/BSON
Almacenamiento principal	RAM	Disco	Disco
Velocidad	Muy alta	Media	Alta
JOINS	No	Si	No (usa <code>\$lookup</code>)
Esquema fijo	No	Si	No
Consultas complejas	Limitadas	Muy potentes	Potentes
Caso típico	Caché, sesiones, rankings	Sistemas transaccionales	Aplicaciones documentales

Almacenamiento de los Datos

Redis almacena los datos principalmente en **RAM**.

Ventajas

- Acceso extremadamente rápido.
- Latencia muy baja.
- Ideal para tiempo real.

Desventajas

- La RAM es limitada y más costosa.
- Si no se configura persistencia, los datos pueden perderse ante una caída.

Tipos de Datos

Tipo	Descripción	Ejemplo de uso
String	Texto o números	Sesiones, contadores
Hash	Objeto clave-valor	Perfil de usuario
List	Lista ordenada	Cola de tareas
Set	Conjunto sin repetidos	Tags, categorías
Sorted Set (ZSet)	Conjunto ordenado por score	Rankings
Stream	Flujo de eventos	Eventos en tiempo real
JSON	Documentos JSON	Objetos complejos
Geo	Coordenadas geográficas	Mapas, ubicaciones
Vector	Embeddings para IA	Búsqueda semántica

Características Principales

Característica	Descripción
In-Memory	Datos en RAM
Clave-Valor	Acceso mediante clave
Muy rápido	Latencias de milisegundos o microsegundos
Schemaless	Sin esquema fijo
Persistencia opcional	RDB y AOF
Open Source	Código abierto
Multiuso	BD, caché y broker de mensajes

Característica	Descripción
Soporte de estructuras complejas	Hashes, Sets, Streams, etc.

Cuándo usar Redis

Caso	Motivo
Caché	Evita consultas repetidas a BD
Sesiones web	Acceso instantáneo
Carritos de compra	Datos temporales
Rankings	Sorted Sets
Contadores	Incrementos atómicos
Pub/Sub	Comunicación entre servicios
Tokens y códigos temporales	TTL automático
IA y búsqueda vectorial	Almacenamiento de embeddings

Transacciones en Redis

Redis tiene soporte transaccional, pero **no es equivalente al de un RDBMS**.

Garantías

- Atomicidad sobre una clave/agregado.
- Operaciones ejecutadas en orden.
- No hay interferencia durante la ejecución.

Limitaciones

- No maneja relaciones entre múltiples entidades como un RDBMS.
- Durabilidad depende de la persistencia configurada.
- En sistemas distribuidos puede priorizar disponibilidad sobre consistencia.

Persistencia

Redis puede guardar datos en disco mediante dos mecanismos.

RDB (Snapshots)

Guarda una **fotografía completa** de la memoria cada cierto tiempo.

Ventajas

- Archivo pequeño.
- Recuperación rápida.
- Ideal para backups.

Desventajas

- Puede perder datos entre snapshots.

AOF (Append Only File)

Registra cada operación de escritura.

Ventajas

- Mayor durabilidad.
- Menor pérdida de datos.

Desventajas

- Archivo más grande.
- Recuperación más lenta.

Comandos

Strings

Operación	Descripción	Ejemplo
SET	Crea o reemplaza una clave.	<code>SET user "Cronos Turismo"</code>
GET	Obtiene el valor de una clave.	<code>GET user</code>
APPEND	Agrega texto al final del valor.	<code>APPEND user " S.A."</code>
DEL	Elimina una clave.	<code>DEL user</code>
EXISTS	Verifica si una clave existe.	<code>EXISTS visits</code>
INCR	Incrementa en 1 un número.	<code>INCR visits</code>
INCRBY	Incrementa en N unidades.	<code>INCRBY visits 5</code>
DECR	Decrementa en 1.	<code>DECR visits</code>
DECRBY	Decrementa en N unidades.	<code>DECRBY visits 2</code>
INCRBYFLOAT	Incrementa valores decimales.	<code>INCRBYFLOAT saldo 200.5</code>
TYPE	Devuelve el tipo de dato de una clave.	<code>TYPE visits</code>

Manejo de Claves

Operación	Descripción	Ejemplo
KEYS *	Obtiene todas las claves.	KEYS *
KEYS patrón	Busca claves usando comodines.	KEYS v*
RENAME	Renombra una clave.	RENAME package "bariloche package"
RENAMENX	Renombra solo si el destino no existe.	RENAMENX origen destino
FLUSHDB	Elimina todas las claves de la BD actual.	FLUSHDB
TYPE	Consulta el tipo de una clave.	TYPE user

Patrones frecuentes para búsquedas de KEYS

Patrón	Significado	Ejemplo
*	Todo	KEYS *
v*	Empieza con v	KEYS v*
t	Contiene t	KEYS *t*
*age	Termina con age	KEYS *age

TTL

Tiene 3 valores posibles:

- Mayor a 0 es el TTL
- -1 es que existe pero sin expiracion.
- -2 es que no existe.

Operación	Descripción	Ejemplo
TTL	Consulta tiempo restante de vida.	TTL agency
EXPIRE	Agrega expiración en segundos.	EXPIRE agency 30
SET EX	Crea una clave con expiración.	SET agency "Cronos" EX 20
PERSIST	Quita la expiración.	PERSIST agency

Listas

Importante recordar que las listas acá arrancan en 0 (por lo tanto si nos piden la posición N, tenemos que buscar la N-1)

Operación	Descripción	Ejemplo
LPUSH	Inserta por izquierda.	LPUSH pets cat
RPUSH	Inserta por derecha.	RPUSH pets dog
LRANGE	Obtiene elementos.	LRANGE pets 0 -1
LPOP	Extrae izquierda.	LPOP pets
RPOP	Extrae derecha.	RPOP pets
LLEN	Cantidad de elementos.	LLEN pets
LINDEX	Obtiene elemento por índice.	LINDEX pets 2
LINSERT	Inserta antes/después de un valor.	LINSERT lista AFTER brc fte
LSET	Modifica elemento por índice.	LSET lista -1 sla
LREM	Elimina ocurrencias.	LREM lista 1 aep
LTRIM	Conserva un rango.	LTRIM lista 2 4
LPOS	Busca posiciones de un valor.	LPOS lista fte COUNT 0
SORT	Devuelve elementos ordenados.	SORT lista ALPHA

Sets

Operación	Descripción	Ejemplo
SADD	Agrega elementos.	SADD airports eze aep mdz
SMEMBERS	Lista todos los elementos.	SMEMBERS airports
SCARD	Cantidad de elementos.	SCARD airports
SREM	Elimina un elemento.	SREM airports cpc
SPOP	Elimina uno aleatorio.	SPOP airports
SISMEMBER	Verifica pertenencia.	SISMEMBER airports eze
SMOVE	Mueve elemento entre conjuntos.	SMOVE origen destino eze
SUNION	Unión.	SUNION a b
SUNIONSTORE	Guarda la unión.	SUNIONSTORE total a b
SINTER	Intersección.	SINTER a b
SDIFF	Diferencia.	SDIFF a b

Sorted Sets (ZSets)

En estos Sets cada elemento tiene un **score** y se **ordenan automáticamente**.

Operación	Descripción	Ejemplo
ZADD	Agrega elemento con score.	ZADD passengers 2.5 federico
ZRANGE	Lista elementos ordenados.	ZRANGE passengers 0 -1
ZRANGE WITHSCORES	Lista con scores.	ZRANGE passengers 0 -1 WITHSCORES
ZRANGE REV	Orden inverso.	ZRANGE passengers 0 -1 REV
ZINCRBY	Incrementa score.	ZINCRBY passengers 2 alejandra
ZCARD	Cantidad de elementos.	ZCARD passengers
ZCOUNT	Cuenta scores dentro de un rango.	ZCOUNT passengers 2 3
ZRANK	Ranking ascendente.	ZRANK passengers julian
ZSCORE	Obtiene score.	ZSCORE passengers tomas
ZPOPMIN	Extrae el menor score.	ZPOPMIN passengers
ZPOPMAX	Extrae el mayor score.	ZPOPMAX passengers
ZREM	Elimina un miembro.	ZREM passengers silvia

Hashes

Para entenderlo mentalmente los Hashes se tratan como un JSON simple, ejemplo:

```
{
  "razon_social": "Cronos SA",
  "domicilio": "47 236",
  "mail": "info@cronos.com.ar"
}
```

se almacena como:

```
HSET user:cronos razon_social "Cronos SA" domicilio "47 236" mail
"info@cronos.com.ar"
```

Operación	Descripción	Ejemplo
HSET	Agrega campos.	HSET user:1 nombre Juan edad 25

Operación	Descripción	Ejemplo
HGET	Obtiene un campo.	HGET user:1 nombre
HGETALL	Obtiene todos los campos.	HGETALL user:1
HDEL	Elimina un campo.	HDEL user:1 telefono
HLEN	Cantidad de campos.	HLEN user:1
HKEYS	Obtiene nombres de campos.	HKEYS user:1
HVALS	Obtiene valores.	HVALS user:1
HEXISTS	Verifica existencia de un campo.	HEXISTS user:1 mail
HSTRLEN	Longitud de un valor.	HSTRLEN user:1 mail

Geospatial

En tema de unidades se manejan m, km, mi, ft .

Operación	Descripción	Ejemplo
GEOADD	Agrega coordenadas.	GEOADD cities -58.37 -34.61 "Buenos Aires"
GEOPOS	Obtiene coordenadas.	GEOPOS cities "La Plata"
GEODIST	Calcula distancia.	GEODIST cities "BA" "Cordoba" km
GEOSEARCH FROMLONLAT	Busca por coordenadas.	GEOSEARCH cities FROMLONLAT -55.9 -27.37 BYRADIUS 100 km
GEOSEARCH FROMMEMBER	Busca desde una ciudad existente.	GEOSEARCH cities FROMMEMBER "Cordoba" BYRADIUS 700 km